

Requêtes additionnelles

Kallouch Iman, Descamps Joseph, De Meester David, Josephy Cedric

Mai 2026

1 Requête 1 — Les 10 utilisateurs ayant le plus de points

SQL

```
SELECT id_user, username, profile_points
FROM users
ORDER BY profile_points DESC
LIMIT 10;
```

Algèbre relationnelle

Impossible en algèbre relationnelle classique car :

- le tri (ORDER BY) n'est pas supporté ;

Calcul tuple

Impossible car le calcul tuple ne permet pas le tri des résultats.

2 Requête 2 — Les utilisateurs ayant publié des résumés dans au moins 3 cours différents

SQL

```
SELECT u.id_user, u.username
FROM users u
JOIN summaries s ON u.id_user = s.user_id
GROUP BY u.id_user, u.username
HAVING COUNT(DISTINCT s.course_id) >= 3;
```

Algèbre relationnelle

$$\pi_{S1.user_id}(\sigma_{S1.user_id=S2.user_id \wedge S2.user_id=S3.user_id \wedge S1.course_id \neq S2.course_id}(\rho_{S1}(summaries) \times \rho_{S2}(summaries)))$$

Calcul tuple

$$\{u \mid users(u) \wedge \exists s_1 \exists s_2 \exists s_3 (summaries(s_1) \wedge summaries(s_2) \wedge summaries(s_3) \\ \wedge s_1.user_id = u.id_user \wedge s_2.user_id = u.id_user \wedge s_3.user_id = u.id_user \\ \wedge s_1.course_id \neq s_2.course_id \wedge s_1.course_id \neq s_3.course_id \wedge s_2.course_id \neq s_3.course_id)\}$$

3 Requête 3 — Le cours ayant le plus de résumés publiés

SQL

```
SELECT c.id_course, c.course_title, COUNT(s.id_summary) AS nb_summaries
FROM courses c
JOIN summaries s ON c.id_course = s.course_id
GROUP BY c.id_course, c.course_title
ORDER BY nb_summaries DESC
LIMIT 1;
```

Algèbre relationnelle

Compter le nombre de résumés et le maximum ne peuvent pas être exprimés en algèbre relationnelle classique.

Calcul tuple

Impossible car le calcul tuple ne supporte pas les fonctions d'agrégation telles que COUNT ou MAX.

4 Requête 4 — Les résumés les mieux notés pour chaque cours

SQL

```
SELECT s1.id_summary, s1.title, s1.course_id, s1.average_rating
FROM summaries s1
WHERE s1.average_rating = (
    SELECT MAX(s2.average_rating)
    FROM summaries s2
    WHERE s2.course_id = s1.course_id
);
ORDER BY s1.course_id ASC;
```

Algèbre relationnelle

$$\begin{aligned} \text{NonMax} &= \pi_{S_1.id_summary, S_1.title, S_1.course_id, S_1.average_rating} (\\ &\sigma_{S_1.course_id=S_2.course_id \wedge S_1.average_rating < S_2.average_rating} (\rho_{S_1}(\text{summaries}) \times \rho_{S_2}(\text{summaries})) \end{aligned}$$

$$\text{Résultat} = \pi_{id_summary, title, course_id, average_rating}(\text{summaries}) - \text{NonMax}$$

Calcul tuple

$$\{s \mid \text{Summaries}(s) \wedge \neg \exists t (\text{Summaries}(t) \wedge t.course_id = s.course_id \wedge t.average_rating > s.average_rating)\}$$

5 Requête 5 — Les utilisateurs n’ayant jamais publié de résumé

SQL

```
SELECT u.id_user, u.username
FROM users u
LEFT JOIN summaries s ON u.id_user = s.user_id
WHERE s.id_summary IS NULL;
```

Algèbre relationnelle

$$users - \pi_{users.*}(users \bowtie summaries)$$

Calcul tuple

$$\{u \mid users(u) \wedge \neg \exists s (summaries(s) \wedge s.user_id = u.id_user)\}$$

6 Requête 6 — L’objet cosmétique le plus acheté

SQL

```
SELECT c.id_item, c.name, COUNT(*) AS purchases_count
FROM cosmetic_items c
JOIN transactions t ON c.id_item = t.item_id
WHERE t.transaction_type = 'purchase_item'
GROUP BY c.id_item, c.name
ORDER BY purchases_count DESC
LIMIT 1;
```

Algèbre relationnelle

Compter le nombre d’objets achetés et le maximum ne peuvent pas être exprimés en algèbre relationnelle classique.

Calcul tuple

Impossible car le calcul tuple ne supporte pas les fonctions d'agrégation.

7 Requête 7 — Les utilisateurs ayant dépensé plus de points qu'ils n'en possèdent actuellement

SQL

```
SELECT u.id_user, u.username, u.profile_points, SUM(ABS(t.amount)) AS total_spent
FROM users u
JOIN transactions t ON u.id_user = t.user_id
WHERE t.transaction_type = 'purchase_item'
GROUP BY u.id_user, u.username, u.profile_points
HAVING SUM(ABS(t.amount)) > u.profile_points;
```

Algèbre relationnelle

Possible uniquement avec une algèbre relationnelle étendue supportant les fonctions d'agrégation telles que SUM.

Calcul tuple

Impossible car le calcul tuple ne supporte pas les fonctions d'agrégation.

8 Requête 8 — Le nombre moyen de résumés publiés par utilisateur

SQL

```
SELECT AVG(summary_count)
FROM (
    SELECT u.id_user, COUNT(s.id_summary) AS summary_count
    FROM users u
    LEFT JOIN summaries s ON u.id_user = s.user_id
    GROUP BY u.id_user
) AS stats;
```

Algèbre relationnelle

Possible uniquement avec une algèbre relationnelle étendue supportant les fonctions d'agrégation telles que COUNT et AVG.

Calcul tuple

Impossible car le calcul tuple ne supporte pas les fonctions d'agrégation.